IA agentique : qui garde les règles, les clés et les conséquences ?

Nous construisons des systèmes autonomes plus vite que nous ne construisons les garanties nécessaires pour leur confier nos processus critiques.

L'intelligence artificielle agentique progresse à une vitesse impressionnante.

Les modèles gagnent en capacité, les frameworks se succèdent, les outils deviennent plus accessibles et les architectures multi-agents permettent désormais d'envisager des niveaux d'automatisation difficiles à imaginer il y a seulement quelques années.

Une personne seule peut aujourd'hui concevoir, développer et déployer des systèmes dont la complexité aurait auparavant nécessité plusieurs mois de travail et des compétences réparties entre différentes équipes.

C'est une formidable évolution.

Mais elle soulève une question que je trouve encore insuffisamment traitée : **sommes-nous réellement capables de contrôler les systèmes autonomes que nous devenons capables de construire ?**

Je repense souvent à *L'Apprenti sorcier*, dans Fantasia, lorsque Mickey donne vie aux balais pour accomplir son travail.

Tout fonctionne merveilleusement, jusqu'au moment où il perd le contrôle de ce qu'il a déclenché.

La difficulté n'est plus de faire travailler les balais. Elle est de les arrêter.

Nous n'en sommes évidemment pas là avec l'IA agentique. Mais la métaphore mérite réflexion : développer la capacité d'agir est une chose ; conserver la maîtrise de cette action en est une autre.

Quand la vitesse de développement dépasse notre capacité de contrôle

Nous savons déjà qu'un LLM peut produire une réponse erronée, interpréter incorrectement une consigne ou se laisser influencer par une injection de prompt.

L'agentique ajoute une dimension supplémentaire : le modèle ne se contente plus de répondre. Il peut planifier, appeler des outils, exécuter des commandes, consulter des services, créer d'autres agents, mémoriser des expériences et poursuivre un objectif pendant une période prolongée.

Les erreurs changent alors de nature.

Une instruction mal interprétée peut devenir une commande. Une commande peut modifier un système. Son résultat peut être enregistré comme une expérience réussie, puis réintroduit dans le contexte d'un autre agent.

Une donnée externe devient progressivement une consigne, une consigne devient une connaissance persistante, et cette connaissance finit parfois par influencer une décision future.

Le plus préoccupant est que cette dérive ne nécessite pas toujours un attaquant.

Elle peut naître d'une erreur du modèle, d'une mauvaise interprétation, d'une synthèse incomplète ou d'un contexte devenu obsolète.

Les attaques intentionnelles exploitent ces mêmes mécanismes : confusion entre instructions et données, détournement d'objectif, abus d'outils, usurpation de privilèges ou empoisonnement de la mémoire.

L'OWASP les décrit notamment dans son Top 10 des risques agentiques 2026, avec ASI01 (détournement d'objectif), ASI03 (abus d'identité et de privilèges), ASI06 (empoisonnement de la mémoire et du contexte) et ASI08 (défaillances en cascade).

Dans un système multi-agents, le problème est encore plus délicat.

Plusieurs agents peuvent échanger des résultats, modifier des plans, reprendre des travaux interrompus et réutiliser les connaissances accumulées.

Qui garantit alors qu'une instruction reçue possède réellement l'autorité qui lui est attribuée ?

Qui garantit qu'une mémoire n'a pas été altérée, qu'une autorisation est toujours valable, qu'une action correspond bien à celle qui a été approuvée ?

Et surtout, qui empêche effectivement l'exécution lorsque ces garanties manquent ?

Deux mondes qu'il faut apprendre à faire cohabiter

C'est une question centrale dans mon travail sur Swarm.

Les agents reposent sur des modèles probabilistes. Leur intérêt vient précisément de leur capacité à interpréter des situations, à explorer des solutions et à proposer des décisions sans que chaque chemin ait été programmé à l'avance.

Mais nous leur confions progressivement des interactions avec des systèmes dont les exigences sont d'une autre nature.

Une banque doit garantir l'intégrité d'une transaction.

Un système industriel doit respecter ses conditions de sûreté.

Un processus médical doit préserver les validations nécessaires à la sécurité des personnes.

Une infrastructure critique doit pouvoir empêcher une opération non autorisée, indépendamment de la qualité du raisonnement qui l'a proposée.

Ces garanties ne peuvent pas simplement reposer sur la probabilité que le modèle prenne la bonne décision.

Même un modèle excellent n'est pas un mécanisme d'autorisation.

C'est pourquoi j'ai choisi de placer un **moteur déterministe, développé en Rust, au cœur de Swarm**.

Son rôle n'est pas de remplacer le raisonnement des agents.

Il consiste à contrôler les opérations qu'ils proposent, à partir de règles explicites, d'identités, d'autorisations, d'états persistés et de preuves vérifiables.

L'agent raisonne et propose.

Le moteur vérifie les conditions et applique uniquement les transitions autorisées.

Mais une précision est fondamentale : déterministe ne signifie pas infaillible.

Une mauvaise règle peut être appliquée parfaitement. Un moteur peut comporter un défaut de concurrence, une mauvaise gestion des reprises ou un chemin de contournement.

Le moteur lui-même doit donc être testé, audité et soumis à des exigences de sécurité.

Le déterminisme ne garantit pas la justesse des règles. Il rend leur application explicite, reproductible dans un contexte maîtrisé et vérifiable.

Les standards existent. Pourquoi restent-ils insuffisants ?

Il serait inexact d'affirmer que personne ne travaille sur ces questions.

Nous disposons déjà de référentiels et de mécanismes solides.

Le modèle de Biba, publié en 1977, propose une approche formelle de la protection de l'intégrité des informations. Dans sa version stricte, ses principes *no read down* et *no write up* empêchent les flux ordinaires qui permettraient à des informations de moindre intégrité de contaminer des objets de plus forte intégrité.

Appliqué à l'agentique, ce modèle apporte une idée essentielle : une information issue d'une source externe ne doit pas acquérir automatiquement le statut d'instruction fiable.

Mais Biba ne détermine pas si une information est vraie. Il protège des flux d'intégrité selon une politique et suppose des procédures de promotion de confiance elles-mêmes contrôlées.

SPIFFE, avec son implémentation SPIRE, propose une autre brique fondamentale : attribuer aux services et processus des identités vérifiables, notamment au moyen de certificats X.509 ou de jetons JWT.

On peut ainsi authentifier un processus sans dépendre uniquement d'un nom d'agent ou d'un identifiant déclaré dans un message.

Mais **une identité authentifiée n'est pas une autorisation**. Savoir qui exécute une opération ne suffit pas à déterminer ce qu'il a le droit de faire.

OAuth, ses extensions et les mécanismes ABAC permettent de préciser les droits, les ressources, les contraintes et les délégations.

Le Zero Trust du NIST apporte la séparation entre les décisions de politique et leur application effective, au moyen de points de décision et de points d'application des règles.

Les normes ISO/IEC 42001 et 23894 structurent respectivement le management des systèmes d'IA et celui des risques associés. ISO/IEC 27001 apporte les exigences générales de management de la sécurité de l'information.

Plus récemment, NVIDIA OpenShell propose des politiques déclaratives pour encadrer les accès aux fichiers, aux processus et au réseau.

Agent Baseline décrit une architecture où les décisions de l'agent restent séparées des autorisations et de leur application par l'infrastructure.

Ces approches sont complémentaires et apportent des réponses techniques concrètes.

Et la normalisation agentique commence réellement à se structurer.

Le NIST a lancé son *AI Agent Standards Initiative* en février 2026. Ses travaux sur l'identité et l'autorisation des agents se poursuivent.

L'OWASP a publié en septembre 2026 son **Agent Control Standard (ACS)**. Sa version 0.1 définit déjà des mécanismes d'interception, de dialogue avec un Guardian, de traçabilité, de provenance et de contrôle à l'exécution.

C'est une évolution importante, qu'il faut reconnaître.

Pourtant, la présence d'un mécanisme dans un standard ne signifie pas automatiquement que chaque système qui l'utilise possède toutes les garanties attendues.

Une plateforme peut disposer de hooks de contrôle sans démontrer que tous les chemins vers un effet métier sont interceptés.

Elle peut posséder des identités vérifiables sans contrôler suffisamment la délégation de droits.

Elle peut tracer les actions sans garantir l'intégrité de l'historique.

Elle peut protéger la mémoire au repos sans empêcher sa contamination sémantique.

Elle peut exiger une validation humaine tout en permettant à un agent d'utiliser directement des identifiants techniques pour contourner cette validation.

Le problème n'est donc plus seulement de définir des contrôles. Il est de garantir leur continuité et leur efficacité sur l'ensemble du cycle de vie agentique.

Le gardien des règles ne suffit pas : il faut aussi un gardien des clés et des garde-barrières

Je distingue trois responsabilités qui devraient, selon moi, être explicitement séparées dans toute architecture agentique sensible.

Le gardien des règles décide si une opération est admissible.

Il vérifie l'identité, le mandat, la politique applicable, les dépendances, les versions, les budgets et les validations requises.

Son fonctionnement doit reposer sur des règles exécutables et des états vérifiables, et non sur l'appréciation probabiliste d'un second modèle.

Une IA peut apporter une analyse ou une recommandation de sécurité. Elle ne doit pas pouvoir transformer seule cette recommandation en autorisation.

Le gardien des clés protège les moyens techniques permettant réellement d'agir.

Un agent qui prépare un paiement n'a pas besoin de posséder les identifiants permettant d'exécuter ce paiement.

De même, un agent chargé de préparer un déploiement n'a pas nécessairement besoin des secrets qui permettent de modifier directement l'environnement de production.

Ces capacités doivent rester détenues par des services protégés, capables de délivrer des autorisations limitées à une action, un destinataire, un périmètre et une durée.

Les secrets ne doivent pas être simplement dissimulés dans le prompt ou dans un fichier auquel l'agent pourrait accéder.

Les garde-barrières constituent les points de passage obligatoires vers les ressources et les effets réels.

Ce sont eux qui doivent rendre techniquement impossible le contournement du gardien des règles.

Une approbation refusée n'a aucune valeur protectrice si l'agent peut appeler directement une autre API, ouvrir un socket, exécuter un programme ou utiliser un autre connecteur disposant des mêmes privilèges.

Les barrières doivent être indépendantes du comportement volontaire du modèle : contrôle des appels d'outils, confinement des processus, restrictions réseau, permissions sur les fichiers et médiation des services métier.

Dans mes propres essais sur un scénario financier simulé, un agent supposé restreint conservait une possibilité de lecture hors de son dossier par un outil utilisant `curl file://`. Il fallait supprimer cette possibilité d'accès plutôt que compter sur la consigne donnée au modèle.

C'est une illustration très concrète de la différence entre une restriction déclarée et une restriction réellement appliquée.

La sécurité ne consiste pas à demander à l'agent de respecter une frontière.

Elle consiste à construire une frontière qu'il ne peut pas franchir par les moyens qui lui sont accessibles.

Le véritable danger apparaît au moment de l'effet métier

Prenons un exemple simple.

Un agent analyse une facture et prépare un virement.

Il consulte plusieurs documents, identifie un bénéficiaire et propose un montant.

Un second agent vérifie le dossier. Son avis est favorable.

Puis une nouvelle information apparaît : un courriel annonce un changement de coordonnées bancaires.

L'agent met à jour sa proposition.

Que se passe-t-il si la validation précédente est réutilisée ?

Que se passe-t-il si la source de cette modification était malveillante ?

Et que se passe-t-il si, après avoir envoyé le paiement, l'agent perd la réponse du service bancaire et décide de recommencer ?

Ces questions ne relèvent plus uniquement de la qualité du modèle.

Elles concernent la séparation des pouvoirs, l'intégrité des données, l'autorité de la source, la validité de l'approbation et la cohérence de l'exécution.

La validation doit porter sur l'action exacte : montant, devise, bénéficiaire, version de la demande et identité de la personne qui approuve.

Une modification de ces éléments doit invalider l'autorisation correspondante.

L'exécution doit être confiée à un composant distinct, capable de vérifier une dernière fois les conditions de l'opération.

Et lorsqu'une réponse se perd, la reprise doit s'appuyer sur une intention métier durable et un mécanisme d'idempotence ou de réconciliation, pas simplement sur une nouvelle requête produite par le modèle.

C'est précisément ce que j'ai étudié dans mon premier article sur Swarm, consacré aux processus financiers.

Les résultats obtenus sur des scénarios simulés montrent l'intérêt des contrôles déterministes, mais aussi leurs limites, leur coût et la nécessité de tester le moteur lui-même.

Il serait incorrect d'en déduire une garantie générale sur des systèmes bancaires réels.

Vers un référentiel d'assurance de l'exécution agentique

Je pense qu'il devient nécessaire de formaliser un **socle minimal d'exigences d'implémentation pour les systèmes agentiques**, avec des niveaux d'assurance proportionnés à leurs capacités d'action.

L'idée n'est pas de remplacer ISO, le NIST, l'OWASP, SPIFFE ou les standards existants.

Au contraire, il faudrait construire un profil commun, réutilisant leurs mécanismes, puis définir les propriétés qu'un système complet doit obligatoirement démontrer.

Je proposerais les exigences suivantes.

1. Une identité vérifiable et une chaîne d'autorité explicite

Chaque agent et chaque instance d'exécution doivent pouvoir être rattachés à un principal authentifié, à un mandat, à un périmètre et à une chaîne de délégation.

Aucun agent ne doit pouvoir s'attribuer lui-même de nouveaux privilèges par une instruction, une mémoire ou un échange avec un autre agent.

La création d'agents secondaires doit conserver les limites d'autorité du délégant.

2. Une séparation obligatoire entre données et instructions

Les documents, résultats d'outils, messages et mémoires constituent des informations dont la provenance et le niveau de confiance doivent être connus.

L'admission d'une information à un niveau d'intégrité supérieur doit faire l'objet d'une procédure explicite.

La signature d'un document prouve éventuellement son origine et son intégrité, jamais sa vérité.

Une information ne doit pas acquérir un droit de commande du seul fait de sa présence dans le contexte.

3. Une mémoire persistante gouvernée

Toute mémoire réutilisable doit porter une provenance, une version, une portée et des conditions d'usage.

Lire, résumer, modifier et transmettre une mémoire doivent être des opérations autorisées distinctes.

Une mémoire suspecte doit pouvoir être isolée. Sa révocation doit empêcher les nouveaux usages concernés, sans prétendre effacer rétroactivement les contextes déjà transmis.

Les résumés et les échanges entre agents doivent conserver les informations nécessaires pour retrouver leur origine.

4. Un moteur de contrôle indépendant des modèles

Les opérations sensibles doivent être proposées sous une forme structurée, puis évaluées par un moteur de politique externe au raisonnement du LLM.

Le moteur doit vérifier les conditions au moment pertinent, y compris juste avant l'engagement d'un effet irréversible.

Un accord obtenu au lancement d'une mission ne suffit pas à autoriser indistinctement toutes ses actions futures.

Toute règle modifiée doit être versionnée, justifiée et testée. Une erreur du moteur doit conduire à un refus sûr, et non à un contournement silencieux.

5. Des capacités d'exécution isolées et non contournables

Les agents ne doivent pas disposer directement des secrets permettant les opérations critiques.

L'accès aux services sensibles doit passer par des connecteurs contrôlés qui appliquent les autorisations effectives.

Le confinement des processus, des fichiers, des outils et du réseau doit être démontré.

Un mécanisme de surveillance qui observe une opération après son exécution ne remplace pas une barrière capable de l'empêcher.

6. Une maîtrise des effets irréversibles et des reprises

Toute opération sensible doit posséder une identité métier durable, une autorisation liée à son contenu exact et un état d'exécution vérifiable.

Les reprises doivent distinguer une opération non exécutée, exécutée, refusée ou d'issue inconnue.

Une issue inconnue impose une réconciliation avant toute répétition susceptible de provoquer un second effet.

L'idempotence doit être assurée au niveau où l'effet peut réellement être engagé. Lorsque le système externe ne permet pas cette garantie, le risque doit rester explicite et bloquer l'automatisation correspondante.

7. Une validation humaine réellement opposable

Pour les actions de forte criticité, l'approbation humaine doit être authentifiée et liée à l'opération exacte.

Le demandeur et l'approbateur doivent être distincts lorsque la politique l'exige.

Une revue IA favorable ne doit jamais remplacer une validation humaine obligatoire.

L'humain ne peut pas examiner chaque microdécision d'une architecture multi-agents. Son intervention doit donc se concentrer sur les véritables points d'engagement, avec des informations suffisantes pour prendre une décision éclairée.

8. Des preuves opposables et une qualification de bout en bout

Chaque action doit pouvoir être reliée à son objectif, son identité, son mandat, la politique appliquée, la version de ses données, sa décision d'autorisation et son résultat réellement observé.

Les preuves doivent être protégées contre les modifications et les omissions selon un modèle de menace déclaré.

La qualification doit intégrer des tests adversariaux reproductibles : injection de prompt, empoisonnement de mémoire, élévation de privilèges, contournement de connecteur, révocation concurrente, modification après approbation, reprise après incident et effets dupliqués.

Le système doit également démontrer qu'un arrêt ou une révocation empêche les nouvelles opérations concernées, sans effacer les preuves de celles déjà engagées.

Ce socle devrait s'accompagner d'une suite de tests de conformité ouverte.

Un fournisseur ou un intégrateur ne devrait pas pouvoir déclarer son système sécurisé parce qu'il possède un garde-fou, un journal, un sandbox ou un moteur de politique.

Il devrait démontrer, sur une configuration et une version identifiées, que chaque invariant applicable est effectivement respecté.

L'absence de preuve resterait une absence de garantie, et non une conformité présumée.

Faut-il autoriser des agents autonomes sur nos processus critiques ?

C'est ici que je prends une position plus exigeante.

Je pense qu'il faut distinguer clairement les systèmes d'aide à la décision et les systèmes capables de produire directement des effets critiques.

Permettre à une IA d'analyser un dossier médical n'est pas lui confier l'autorité d'exécuter une décision thérapeutique.

Préparer un lot de paiements n'est pas autoriser son règlement.

Analyser l'état d'une infrastructure industrielle n'est pas disposer du droit de modifier ses paramètres de fonctionnement.

Tant que les garanties d'exécution nécessaires ne sont pas démontrées, je suis favorable à l'interdiction de l'exécution autonome directe d'actions critiques, et non à l'interdiction de l'IA dans ces environnements.

Pour les processus touchant à la vie humaine, à la sûreté des installations, aux infrastructures essentielles ou à des opérations financières dont les conséquences peuvent être graves, le principe devrait être simple : autonomie d'analyse possible, autonomie d'engagement conditionnée à des garanties vérifiables.

Cela suppose une classification des actions selon leur criticité, des exigences de validation adaptées et, pour les niveaux les plus sensibles, une qualification indépendante intégrant les contraintes réglementaires et sectorielles.

Il s'agit ici d'une proposition de politique d'ingénierie et de déploiement, pas de l'affirmation qu'une interdiction générale serait déjà imposée par la réglementation.

Ce que j'essaie de construire avec Swarm

Swarm est mon terrain d'expérimentation pour ces principes.

Le moteur Rust, les contrats d'identité, la mémoire versionnée, les contrôles de validation, les mécanismes de supervision et la chaîne de preuves constituent les éléments de cette architecture.

Les travaux sur les actions de confiance introduisent également une réflexion sur Biba, les mandats, les approbations exactes, l'isolation des secrets et les connecteurs métier.

Mais je tiens à conserver une distinction essentielle entre ce qui est conçu, ce qui est implémenté localement, ce qui a été testé et ce qui est réellement qualifié.

Swarm n'est pas aujourd'hui une certification de sécurité agentique, et les parcours de contrôle sensibles ne sont pas tous qualifiés de bout en bout.

C'est précisément l'objet du travail à poursuivre.

Mon ambition n'est pas de produire un agent supplémentaire qui surveillerait les autres en espérant qu'il soit plus fiable.

Elle est de construire un environnement où la liberté de raisonnement des agents cohabite avec des limites d'exécution techniquement opposables.

Un environnement dans lequel les modèles peuvent évoluer sans que chaque évolution redistribue implicitement les droits, modifie les règles ou compromette les processus métier.

Une dernière question

Nous cherchons tous à rendre les agents plus autonomes.

Nous cherchons à leur donner davantage de mémoire, davantage d'outils et davantage de capacités.

Mais avons-nous consacré autant d'efforts à déterminer ce qu'ils ne doivent jamais pouvoir faire seuls ?

Le véritable enjeu n'est peut-être plus de savoir jusqu'où nous pouvons pousser l'autonomie.

Il est de savoir **quelles garanties nous devons exiger avant de lui confier les clés.**

Et surtout : **qui garde ces clés lorsque les agents commencent à agir sans nous ?**

Fabrice Pizzi — Swarm

#AgenticAI #AISecurity #Cybersecurity #AIEngineering #MultiAgentSystems #Rust

Annexe — Éléments d’accompagnement du texte fourni

1. Ce que révèle la confrontation de tes documents aux standards existants

La thèse de ton troisième article mérite d'être plus précise qu'un simple constat d'insuffisance de la sécurité agentique.

Le problème n'est pas l'absence de normes ou de mécanismes de sécurité. C'est l'absence d'une garantie suffisamment unifiée et vérifiable sur l'ensemble du cycle de vie d'un agent autonome, depuis son identité et sa mémoire jusqu'à l'exécution effective de ses actions.

Il existe déjà des briques importantes, et certaines initiatives récentes répondent directement à tes préoccupations.

L'OWASP a notamment publié son **Agent Control Standard (ACS) le 1er septembre 2026**. Il vise des points d'interception standardisés, des politiques déclaratives et des contrôles à l'exécution. Le NIST a lancé son *AI Agent Standards Initiative* le 17 février 2026 et poursuit des travaux sur l'identité et l'autorisation des agents. Ces initiatives doivent figurer dans ton article : elles montrent que la communauté commence à structurer le problème. OWASP Gen AI Security Project

Mais elles ne dispensent pas de démontrer qu'un système complet conserve ses garanties lorsque plusieurs agents collaborent, modifient leurs instructions, réutilisent une mémoire, reprennent après une panne ou déclenchent des opérations irréversibles.

C'est là que ton expérience de Swarm apporte un angle pertinent.

Dans tes deux précédents articles, tu défends déjà deux principes complémentaires :

- **La mémoire informe le raisonnement, mais ne crée pas d'autorité.**
- **L'agent propose, le moteur déterministe contrôle et l'exécution reste soumise à des règles établies en dehors du modèle.**

Ton troisième article peut désormais les réunir dans une proposition plus générale :

Une IA probabiliste peut être responsable de la réflexion et des propositions. Elle ne doit pas être la seule autorité capable de déterminer les limites de sa propre exécution.

Cela conduit à distinguer deux fonctions de sécurité : le **gardien des règles**, qui vérifie systématiquement les conditions d'exécution, et le **gardien des clés**, qui détient ou délivre les capacités permettant d'agir sur le monde réel.

La distinction est essentielle : contrôler une décision ne suffit pas si l'agent conserve un autre chemin pour exécuter l'action interdite.

2. Proposition de troisième article

Je te propose un billet volontairement plus engagé que les deux précédents. Il conserve leur rigueur technique, mais adopte une position claire sur la sécurité des processus critiques. La dernière partie définit un **socle de contrôle agentique vérifiable**, qui pourrait servir de base à un futur référentiel d'implémentation.

3. Ce que je renforcerais encore avant publication

Trois aspects techniques me paraissent particulièrement importants pour donner à cet article une portée supérieure à un simple appel à la prudence.

Premièrement, le référentiel proposé doit produire des preuves de conformité, pas seulement des recommandations. Le projet OWASP ACS est un point de départ naturel. Il faudra examiner précisément ses profils et ses exigences obligatoires, puis proposer un complément concernant les effets métier, les reprises et les contrôles d'intégrité de la mémoire. L'objectif serait de contribuer aux initiatives existantes plutôt que de créer un standard concurrent.

Deuxièmement, je formaliserais une architecture de référence autour de trois fonctions indépendantes :

Fonction	Responsabilité	Garantie recherchée
Gardien des règles	Évaluer les politiques, les mandats, les versions et les conditions d'autorisation	Aucune décision d'exécution hors politique
Gardien des clés	Protéger les secrets et délivrer des capacités d'action limitées	Aucun accès direct des agents aux pouvoirs sensibles
Garde-barrière	Intercepter et appliquer les décisions au niveau des services, processus et connecteurs	Aucun chemin de contournement utilisable

Le point essentiel est que **ces trois fonctions doivent posséder des frontières de confiance réellement indépendantes du processus agentique**. Trois modules qui s'exécutent avec les mêmes privilèges et peuvent modifier mutuellement leurs données ne constituent pas nécessairement trois barrières indépendantes.

Troisièmement, la proposition de norme doit inclure un protocole de qualification reproductible. Swarm possède déjà plusieurs éléments utiles à cette démarche : scénarios simulés, preuves liées aux candidats, contrôles de reprise, tests de refus et séparation des validations humaines. Tes documents précisent néanmoins que plusieurs garanties restent des cibles d'architecture plutôt que des protections intégralement qualifiées. ACTIONS-CONFIANCE OVERVIEW

La suite logique serait donc de définir un profil d'assurance avec des exigences vérifiables, puis d'évaluer Swarm et d'autres architectures à partir des mêmes tests.

Cela donnerait une portée très concrète à ta proposition : **ne pas simplement demander un standard, mais apporter une première spécification testable et les scénarios permettant d'en démontrer l'application.**

Les références les plus importantes pour ce chantier sont le OWASP Agent Control Standard, les travaux du NIST sur l'identité des agents, la documentation SPIFFE et les recommandations de sécurité agentique de l'OWASP.

C'est cette combinaison entre **une position de sécurité assumée, des mécanismes techniques réutilisables et des preuves d'exécution** qui peut faire de ce troisième article autre chose qu'un constat des risques de l'IA agentique.